

Designing a Distributed Rate Limiter

Goal. Throttle how many requests a client (user, IP, API key, or tenant) may make in a time window, protecting downstream services from overload, abuse, and runaway cost — while adding negligible latency and staying correct across many distributed nodes.

01 Clarify requirements

FUNCTIONAL

- Limit requests per client per unit time (e.g. 100 req / min).
- Rules configurable per API route, method, and client tier.
- Reject over-limit requests clearly (HTTP 429) with retry guidance.
- Accurate enough that legitimate traffic is rarely blocked.

NON-FUNCTIONAL

- **Low latency** — a few ms; sits on every request's hot path.
- **High availability** — must not become a single point of failure.
- **Scalable** — millions of RPS across regions.
- **Fault tolerant** — a limiter outage must not take down the API.

Key clarifying questions to ask the interviewer: Client vs. server-side limiting? What identifies a client (IP, user ID, API key)? One rule or per-endpoint rules? Strict accuracy vs. approximate-but-fast? Single datacenter or global? Should limits be exact or is a small burst acceptable?

02 Back-of-the-envelope

10Mpeak requests / sec,
global**~2**counter ops per
request (read + write)**~100M**

active clients tracked

~2 GBcounter state ($\approx 20 \text{ B} \times$
100M keys)

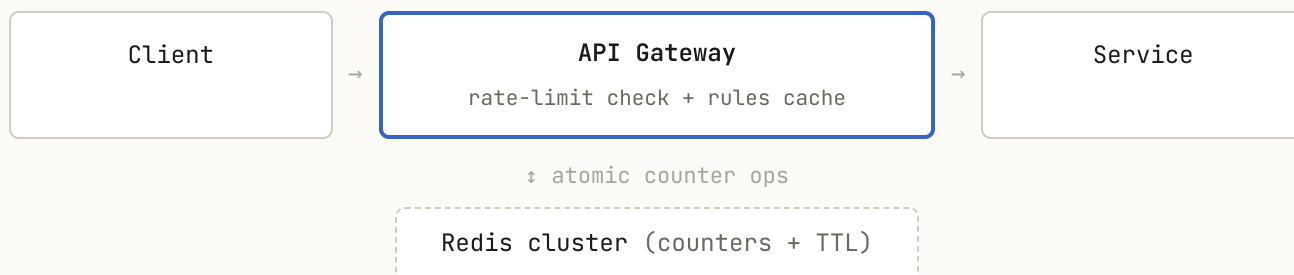
State is tiny and TTL-bounded, so it fits comfortably in memory — this is why an **in-memory store like Redis** is the natural backend. The hard part is not storage; it's doing $\sim 20\text{M}$ atomic counter ops/sec with millisecond latency and keeping counts consistent across nodes.

03 Where does the limiter live?

LOCATION	PROS	CONS
Client-side	No server load	Trivially bypassed — never trust it as the only layer
API Gateway	Central choke point; keeps logic out of services; off the hot code path	Another hop; must scale with total traffic
Middleware / library	In-process, lowest latency; per-service rules	Duplicated across services; coupled to app runtime
Dedicated service	Reusable, independently scalable, single source of truth	Network call per check; adds a dependency

Recommended: a rate-limiting component *at the API gateway / reverse proxy*, backed by a shared distributed store. It's the natural chokepoint every request already passes through, keeps business services clean, and scales horizontally.

FIG 1 • REQUEST FLOW



Allowed → forwarded to the service. Over limit → gateway returns **429** immediately; the request never reaches the backend.

04 Algorithms & tradeoffs

ALGORITHM	MEMORY	BURSTS	ACCURACY	NOTES
Token bucket	$O(1)$	Allows (up to capacity)	Good	Smooth avg rate + controlled burst. Most popular.
Leaky bucket	$O(1)$	Smooths out	Good	Constant outflow; queue can add latency.
Fixed window	$O(1)$	Edge spikes (2×)	Coarse	Simplest; boundary burst problem.
Sliding window log	$O(N)$ per client	Prevented	Exact	Perfectly accurate but memory-heavy.
Sliding window counter	$O(1)$	Mostly prevented	~99%	Best practical accuracy/cost balance.

Token bucket

Bucket holds up to *capacity* tokens, refilled at a fixed rate. Each request consumes one token; empty bucket → reject. Absorbs bursts up to capacity.

```
refill = (now - last) * rate
tokens = min(cap, tokens + refill)
last = now
if tokens ≥ 1:
    tokens -= 1; return ALLOW
return REJECT
```

State: {tokens, last_refill} — 2 fields.

Leaky bucket

Requests enter a FIFO queue that drains at a constant rate. Full queue → reject. Output is perfectly smooth — good for shaping traffic to a steady downstream rate.

```
leak = (now - last) * rate
level = max(0, level - leak)
last = now
if level + 1 ≤ cap:
    level += 1; return ALLOW
return REJECT
```

Trade-off: no bursts, but queuing adds latency.

Fixed window counter

One counter per client per fixed window (e.g. per minute). Cheap, but a client can send $2 \times \text{limit}$ across a window boundary.

```
key = id + ":" + floor(now/W)
n = INCR key
if n == 1: EXPIRE key W
return n ≤ limit ? ALLOW : REJECT
```

Flaw: boundary burst (Fig 2).

Sliding window log

Store a timestamp per request in a sorted set; drop entries older than the window, then count. Exact, but memory grows with request volume.

```
ZREMRANGEBYSCORE key 0 (now-W)
n = ZCARD key
if n < limit:
    ZADD key now now; return ALLOW
return REJECT
```

Cost: $O(N)$ memory per client.

Sliding window counter ★ recommended default

Blends two fixed windows: weight the previous window's count by how much of it still overlaps the rolling window. $O(1)$ memory, smooths boundary spikes, and is accurate to within $\sim 1\%$ — the standard production choice.

```
elapsed = (now mod W) / W
est = curr + prev * (1 - elapsed)
return est < limit ? ALLOW : REJECT
```

EXAMPLE

Limit 100/min. 30 s into the current minute: previous minute had 84 requests, current has 36.

$est = 36 + 84 \cdot (1 - 0.5) = 78 \rightarrow \text{ALLOW}$

FIG 2 • FIXED-WINDOW BOUNDARY BURST

window 12:00–12:01
100 req, all in last 30 s

window 12:01–12:02
100 req, all in first 30 s

Both windows individually respect the 100/min limit, yet **200 requests land inside a single 60-second span** straddling the boundary. Sliding-window algorithms fix this.

05 Distributed rate limiting

With many gateway nodes, per-node in-memory counters let a client exceed the global limit N -fold. Counters must be **shared**. Two problems dominate: where the state lives, and how to update it atomically.

Shared store — Redis

A central Redis cluster holds all counters. In-memory, single-threaded per shard, with atomic INCR/EXPIRE and native TTLs — a near-perfect fit. Shard by client key so each client maps to one node.

Race conditions

Naïve read-modify-write (GET → check → SET) races under concurrency and undercounts. Fix with an **atomic Lua script** (read + decide + write in one round trip) or INCR + compare. Avoid distributed locks — they kill throughput.

```
-- Atomic fixed-window check (Redis Lua). KEYS[1]=client key, ARGV: limit, window
local n = redis.call('INCR', KEYS[1])
if n == 1 then redis.call('EXPIRE', KEYS[1], ARGV[2]) end
if n > tonumber(ARGV[1]) then return 0 end    -- REJECT
return 1                                     -- ALLOW
```

Latency vs. accuracy — the central trade-off. A synchronous call to central Redis on every request is exact but adds a network round trip. The common optimization is a **two-tier / local-token** model: each node holds a local allowance and periodically syncs with the central store (async, or by pre-fetching a batch of tokens). This trades a small amount of accuracy at the edges for big latency and throughput wins. State whether the interviewer wants strict or approximate limits — it drives this decision.

06 Routing state: sticky sessions vs. hashing

A client's counter must consistently reach the *same* place, or nodes will keep divergent counts. Two ways to guarantee that — pin the client to a node (sticky), or hash the client key to a shard (hashing). They solve the same problem at different layers.

Sticky sessions

The load balancer pins each client to one gateway node (by cookie or source IP), so that node can hold the counter **in local memory** — no shared store on the hot path, lowest possible latency.

- + No network hop per check; simple.
- Uneven load ("hot" clients overload a node); state lost on node failure; rebalancing on scale-up scatters counters; a single client spread across connections can still split.

Hashing to a shard

Any gateway node handles any request, but the **client key is hashed to a specific Redis shard** that owns its counter. Gateways stay stateless; the store is the single source of truth.

- + Even load; nodes are disposable; scales cleanly.
- A store round trip per check; rehashing on resize moves keys — use **consistent hashing** to minimize churn.

Recommendation. Prefer **hashing to a sharded store with consistent hashing** — it keeps gateways stateless, survives node failure, and scales predictably, which matters at millions of RPS. Reserve sticky sessions for the two-tier local-token optimization, where a node's local allowance is a deliberate approximation rather than the source of truth. Consistent hashing (or rendezvous hashing) is the key detail: it remaps only K/N keys when a shard is added or removed, instead of nearly all of them.

07 Responding when the limit is hit

Return **HTTP 429 Too Many Requests**. Tell clients how much budget remains and when to retry, so well-behaved clients back off instead of hammering.

- Include `Retry-After` so clients wait the right amount.
- Expose limit / remaining / reset headers on *every* response, not just 429s.
- Consider soft limits (throttle/queue) vs. hard rejects per tier.
- Optionally drop excess silently for abusive clients to save resources.

```
HTTP/1.1 429 Too Many Requests
Retry-After: 12
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1752710412
```

```
{ "error": "rate_limit_exceeded",
  "retry_after": 12 }
```

08 Scaling

- **Shard Redis** by client key; each client's counter lives on one node → no cross-node coordination.
- **Read replicas** and pipelining to raise throughput.
- **Regional clusters** for global scale; limits enforced per region (or async-reconciled) to avoid cross-region latency.
- **Local caching** of rules and hot counters cuts store traffic.
- Scale gateway nodes horizontally — they're stateless.

09 Failure modes

- **Fail open** — if Redis is down, allow requests rather than block all traffic. Availability > strict limiting for most APIs.
- **Redis HA** — replicas + Sentinel/Cluster failover; a node loss shouldn't lose enforcement.
- **Local fallback** — nodes fall back to approximate in-memory limits during a store outage.
- **Monitor** — track 429 rate, store latency, and rejects to tune limits and spot abuse.

Interview cheat sheet

- ① Clarify: who's limited, strict vs. approximate, single vs. global.
- ② Estimate: state is small → in-memory store (Redis).
- ③ Place it at the gateway, backed by a shared store.
- ④ Pick **sliding-window counter** or **token bucket**; justify vs. alternatives.
- ⑤ Make updates atomic (Lua) — call out race conditions.
- ⑥ Route state via consistent hashing (not sticky sessions).
- ⑦ Name the latency ↔ accuracy trade-off (two-tier sync).
- ⑧ Return 429 + Retry-After / X-RateLimit-* headers.
- ⑨ Fail open; Redis HA; monitor 429s.